

20160701

```
public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exam 2016_07_01 - Exercise 1");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(FloatWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(NullWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class MapperBigData extends Mapper<LongWritable, // Input key type
    Text, // Input value type
    Text, // Output key type
    FloatWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        String[] fields=value.toString().split(",");
        context.write(new Text(fields[1]), new FloatWritable(Float.parseFloat(fields[3])));
    }
}

class ReducerBigData extends Reducer<Text, // Input key type
    FloatWritable, // Input value type
    Text, // Output key type
    NullWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<FloatWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        float sum=0;
        for (FloatWritable value : values) {
            sum=sum+value.get();
        }
        // Check if the current book is a best seller
        // If the book is a best seller, emit the pair (bookid,null)
        if (sum>=1000000)
        {
            context.write(key, NullWritable.get());
        }
    }
}

public class BookId implements Function<String, String> {
    @Override
    public String call(String line) {
        String[] fields=line.split(",");
        return fields[0];
    }
}

public class ComplexCount implements Serializable {
    public int numPurchases;
    public int numCheapPurchases;
}

public class PairCustomerCount implements PairFunction<String, String, ComplexCount> {
    @Override
    public Tuple2<String, ComplexCount> call(String line) {
        ComplexCount localCount;
        String[] fields=line.split(",");
        float price=Float.parseFloat(fields[3]);
        localCount=new ComplexCount();
        localCount.numPurchases=1;
        if (price<10) localCount.numCheapPurchases=1;
        else localCount.numCheapPurchases=0;
        return new Tuple2<String, ComplexCount>(fields[0], localCount);
    }
}

public class Expensive implements Function<String, Boolean> {
    @Override
    public Boolean call(String line) {
        String[] fields=line.split(",");
        float suggestedPrice=Float.parseFloat(fields[3]);
        if (suggestedPrice>30) return true;
        else return false;
    }
}

public class Sum implements Function2<ComplexCount, ComplexCount, ComplexCount> {
    @Override
    public ComplexCount call(ComplexCount count1, ComplexCount count2) throws Exception {
        ComplexCount localCount=new ComplexCount();
        localCount.numCheapPurchases=count1.numCheapPurchases+count2.numCheapPurchases;
        localCount.numPurchases=count1.numPurchases+count2.numPurchases;
        return localCount;
    }
}

public class Propensity implements Function<Tuple2<String, ComplexCount>, Boolean> {
    private double threshold;
    public Propensity(double th) {threshold=th;}
    @Override
    public Boolean call(Tuple2<String, ComplexCount> count) {
        if (((double)count._2().numCheapPurchases)>=((double)count._2().numPurchases)*threshold) return true;
        else return false;
    }
}

public class BookIdFromPurchase implements Function<String, String> {
    @Override
    public String call(String line) {
        String[] fields=line.split(",");
        return fields[1];
    }
}
```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathBooks;
        String inputPathBoughtBooks;
        String outputPathNeverSold;
        String outputPathCheckPurchases;
        double threshold;
        inputPathBooks=args[0];
        inputPathBoughtBooks=args[1];
        threshold=Double.parseDouble(args[2]);
        outputPathNeverSold=args[3];
        outputPathCheckPurchases=args[4];
        // Create a configuration object and set the name of the application
        SparkConf conf=new SparkConf().setAppName("Spark Exam 2016_07_01 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of books.txt
        JavaRDD<String> booksRDD = sc.textFile(inputPathBooks);
        // Select expensive books
        JavaRDD<String> expensiveBooksRDD = booksRDD.filter(new Expensive());
        // Select the ids of the expensive books
        JavaRDD<String> IDsExpensiveBooksRDD = expensiveBooksRDD.map(new BookId());
        // Read the content of boughtBooks.txt
        JavaRDD<String> boughtBooksRDD = sc.textFile(inputPathBoughtBooks).cache();
        // Select the ids of the bought books
        JavaRDD<String> IDsBoughtBooksRDD = boughtBooksRDD.map(new BookIdFromPurchase());
        // Compute the difference (apply subtract) between the ids of the expensive books and
        // those of the bought books. Only the ids of the expensive never-sold books are returned
        JavaRDD<String> ExpensiveNeverSoldRDD = IDsExpensiveBooksRDD.subtract(IDsBoughtBooksRDD);
        // Store the selected users in the output folder
        ExpensiveNeverSoldRDD.saveAsTextFile(outputPathNeverSold);
        System.out.println("Number of expensive never-sold books: "+ ExpensiveNeverSoldRDD.count());
        // *****
        // Exercise 2 - Part B
        // *****
        // Count for each customer the number of purchases and
        // the number of cheap purchases
        // This is a word count-like problem. To improve the efficiency
        // of the application we can associate to each customer a
        // complex counter containing two integer values: num. purchases and num. cheap purchases
        // For each line of BoughtBooks, the program emits one pair
        // key = customerid
        // value = ComplexCount
        //     - numPurchases=1, numCheapPurchases=1 if the purchase is cheap
        //     - numPurchases=1, numCheapPurchases=0 if the purchase is not cheap
        JavaPairRDD<String,ComplexCount> CustomerCountRDD=boughtBooksRDD.mapToPair(new PairCustomerCount());
        // Count the total number of purchases and cheap purchases for each customer
        JavaPairRDD<String,ComplexCount> TotalCustomerCountRDD=CustomerCountRDD.reduceByKey(new Sum());
        // Select the users with a propensity to cheap purchases
        JavaPairRDD<String,ComplexCount> SelectedCustomers=TotalCustomerCountRDD.filter(new Propensity(threshold));
        // Select the keys and store them
        SelectedCustomers.keys().saveAsTextFile(outputPathCheckPurchases);
        // Close the Spark context
        sc.close();
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exam 2016_07_12 - Exercise 1");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }
    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class MapperBigData extends Mapper<LongWritable, // Input key type
    Text, // Input value type
    Text, // Output key type
    IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        String[] fields=value.toString().split(",");
        String zone=fields[2];
        String city=fields[3];
        int totalNumberOfSlots=Integer.parseInt(fields[4]);
        if (city.compareTo("Turin")==0 && totalNumberOfSlots>=20) context.write(new Text(zone), new IntWritable(new Integer(1)));
    }
}

class ReducerBigData extends Reducer<Text, // Input key type
    IntWritable, // Input value type
    Text, // Output key type
    IntWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        int sum=0;
        for (IntWritable value : values) sum=sum+value.get();
        context.write(key, new IntWritable(new Integer(sum)));
    }
}

public class SmallStation implements Function<String, Boolean> {
    public Boolean call(String line) {
        String[] fields=line.split(",");
        int totalNumSlots=Integer.parseInt(fields[4]);
        if (totalNumSlots<5)
            return true;
        else
            return false;
    }
}

public class StationIdFromOccupancy implements Function<String, String> {
    @Override
    public String call(String line) {
        String[] fields=line.split(",");
        String stationId=fields[0];
        return stationId;
    }
}

public class StationCounters implements PairFunction<String, String, Integer> {
    @Override
    public Tuple2<String, Integer> call(String line) {
        String[] fields=line.split(",");
        // fields[0] = stationId
        // fields[5] = numFreeSlots
        int numFreeSlots=Integer.parseInt(fields[5]);
        if (numFreeSlots>=3)
            return new Tuple2<String, Integer>(fields[0], new Integer(0));
        else
            return new Tuple2<String, Integer>(fields[0], new Integer(1));
    }
}

public class Sum implements Function2<Integer, Integer, Integer> {
    @Override
    public Integer call(Integer count1, Integer count2) throws Exception {
        return new Integer(count1 + count2);
    }
}

public class WellSized implements Function<Tuple2<String, Integer>, Boolean> {
    @Override
    public Boolean call(Tuple2<String, Integer> stationCounters) {
        // Check if (number of lines with numFreeSlots<3) is 0
        if (stationCounters._2()==0)
            return true;
        else
            return false;
    }
}

public class ZeroFreeSlots implements Function<String, Boolean> {
    @Override
    public Boolean call(String line) {
        String[] fields=line.split(",");
        int numFreeSlots=Integer.parseInt(fields[5]);
        if (numFreeSlots==0)
            return true;
        else
            return false;
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathStations;
        String inputPathStationsOccupancy;
        String outputPathPartA;
        String outputPathPartB;
        inputPathStations=args[0];
        inputPathStationsOccupancy=args[1];
        outputPathPartA=args[2];
        outputPathPartB=args[3];
        // Create a configuration object and set the name of the application
        SparkConf conf=new SparkConf().setAppName("Spark Exam 2016_07_12 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of Stations.txt
        JavaRDD<String> stationsRDD = sc.textFile(inputPathStations);
        // Select small stations
        JavaRDD<String> smallStationsRDD = stationsRDD.filter(new SmallStation());
        // Select the ids of the small stations
        JavaRDD<String> IDsSmallStationsRDD = smallStationsRDD.map(new StationId());
        // Read the content of StationsOccupancy.txt
        JavaRDD<String> stationsOccupancyRDD = sc.textFile(inputPathStationsOccupancy).cache();
        // Select the lines with numFreeSlots = 0
        JavaRDD<String> criticalLinesRDD = stationsOccupancyRDD.filter(new ZeroFreeSlots());
        // Select the ids of the stations associated with a critical situation
        // i.e., the potential critical stations
        JavaRDD<String> IDsCriticalStationsRDD = criticalLinesRDD.map(new StationIdFromOccupancy());
        // Compute the intersection between IDsSmallStationsRDD and IDsCriticalStationsRDD
        // to obtain the set of potential critical small stations
        JavaRDD<String> SmallPotentialCriticalStationsRDD = IDsSmallStationsRDD.intersection(IDsCriticalStationsRDD);
        // Store the selected stations in the output folder
        SmallPotentialCriticalStationsRDD.saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // Count for each station the number of lines
        // of StationOccupancy.txt associated with that station with numFreeSlots<3
        // This is a word count-like problem.
        // If the count for one station is 0 it means
        // that it has always had at least 3 free slots
        // (based on the historical data stored in StationsOccupancy.txt)
        // For each line of StationsOccupancy.txt, the program emits one pair
        // key = stationId
        // value = 0 if numFreeSlots>=3
        //       = 1 if numFreeSlots<3
        JavaPairRDD<String,Integer> StationsCountersRDD=stationsOccupancyRDD.mapToPair(new StationCounters());
        // Count the total number of lines with numFreeSlots<3 for each station
        JavaPairRDD<String,Integer> TotalStationsCountersRDD=StationsCountersRDD.reduceByKey(new Sum());
        // Select the stations with
        // (number of lines with numFreeSlots<3) == 0
        JavaPairRDD<String,Integer> SelectedStations=TotalStationsCountersRDD.filter(new WellSized());
        // Select the keys and store them
        SelectedStations.keys().saveAsTextFile(outputPathPartB);
        // Close the Spark context
        sc.close();
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exam 2016_09_19 - Exercise 1");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

public class AlwaysPolluted implements Function<Tuple2<String, Integer>, Boolean> {
    @Override
    public Boolean call(Tuple2<String, Integer> stationCounters) {
        // Check if (number of lines with PM2.5<=0PM2.5th_limit) is 0
        if (stationCounters._2()==0)
            return true;
        else
            return false;
    }
}

public class GreaterPM10th implements Function<String, Boolean> {
    private Double PM10th_limit;
    public GreaterPM10th(Double PM10th) {
        PM10th_limit=PM10th;
    }
    @Override
    public Boolean call(String reading) throws Exception {
        // station1,2016/03/07,17,15,29.2,15.1
        String[] fields=reading.split(",");
        String year=fields[1].substring(0, 4);
        Double PM10=Double.parseDouble(fields[4]);
        if (year.equals("2015")==true && PM10>PM10th_limit)
            return true;
        else
            return false;
    }
}

public class HPS implements Function<Tuple2<String, Integer>, Boolean> {
    @Override
    public Boolean call(Tuple2<String, Integer> stationCount) throws Exception {
        if (stationCount._2())>45)
            return true;
        else
            return false;
    }
}

public class StationId implements Function<Tuple2<String, Integer>, String> {
    @Override
    public String call(Tuple2<String, Integer> stationCount) throws Exception {
        return stationCount._1();
    }
}

public class SumOnes implements Function2<Integer, Integer, Integer> {
    @Override
    public Integer call(Integer count1, Integer count2) throws Exception {
        return count1+count2;
    }
}

class MapperBigData extends Mapper<
    LongWritable, // Input key type
    Text, // Input value type
    Text, // Output key type
    IntWritable> { // Output value type

    protected void map(
        LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        String[] fields=value.toString().split(",");
        // monitoringStationId,date,hour,minute,PM10value,PM2.5value
        // station1,2016/03/07,17,15,29.2,15.1
        // fields[0] = monitoringStationId
        // fields[1] = date
        // fields[2] = hour
        // fields[3] = minute
        // fields[4] = PM10value
        // fields[5] = PM2.5value
        String stationid=fields[0];
        String year=fields[1].substring(0,4);
        Double PM10=Double.valueOf(fields[4]);
        Double PM25=Double.valueOf(fields[5]);
        // Emit a pair (stationid,1) if the reading satisfies the
        // constraints year=2013 and PM10>PM2.5
        if (year.equals("2013")==true && PM10>PM25)
        {
            context.write(new Text(stationid), new IntWritable(new Integer(1)));
        }
    }
}

class ReducerBigData extends Reducer<
    Text, // Input key type
    IntWritable, // Input value type
    Text, // Output key type
    IntWritable> { // Output value type

    @Override
    protected void reduce(
        Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        // key = station id
        // values = list of ones
        int count=0;
        for (IntWritable value : values) {
            count=count+value.get();
        }
        // Emit a key (stationid, count) only if count >=30
        if (count>=30) {
            context.write(key, new IntWritable(new Integer(count)));
        }
    }
}

public class Sum implements Function2<Integer, Integer, Integer> {
    @Override
    public Integer call(Integer count1, Integer count2) throws Exception {
        return new Integer(count1 + count2);
    }
}

public class StationIdOne implements PairFunction<String, String, Integer> {
    @Override
    public Tuple2<String, Integer> call(String reading) throws Exception {
        // Emit stationid, 1
        // station1,2016/03/07,17,15,29.2,15.1
        String[] fields=reading.split(",");
        String stationId=fields[0];
        return new Tuple2<String, Integer>(stationId, new Integer(1));
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathReadings;
        String outputPathPartA;
        String outputPathPartB;
        Double pm10th_limit;
        Double pm25th_limit;
        inputPathReadings=args[0];
        outputPathPartA=args[1];
        outputPathPartB=args[2];
        pm10th_limit=Double.parseDouble(args[3]);
        pm25th_limit=Double.parseDouble(args[4]);
        // Create a configuration object and set the name of the application
        SparkConf conf=new SparkConf().setAppName("Spark Exam 2016_09_19 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of ReadingsPerMonitoringStations.txt
        JavaRDD<String> readingsRDD = sc.textFile(inputPathReadings);
        // Select the reading of year 2015 with PM10>PM10th_limit
        JavaRDD<String> readings2015HighPM10RDD = readingsRDD.filter(new GreaterPM10th(pm10th_limit));
        // Map to pairs stationid, 1
        JavaPairRDD<String,Integer> stationOnesPairRDD=readings2015HighPM10RDD.mapToPair(new StationIdOne());
        // Count lines per stations
        JavaPairRDD<String,Integer> stationsCountsRDD=stationOnesPairRDD.reduceByKey(new SumOnes());
        // Select Highly Polluted Stations
        JavaPairRDD<String,Integer> highlyPollutedStations=stationsCountsRDD.filter(new HPS());
        // Select the ids of the stations
        // Store the selected stations in the output folder
        highlyPollutedStations.keys().saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // Count for each station the number of lines of ReadingsPerMonitoringStations.txt
        // associated with that station with
        // PM2.5<=PM2.5th_limit
        // If the value is 0 it means that station is a "always polluted station" because
        // the value of PM2.5 was always above the user provided threshold for that station
        // For each line of ReadingsPerMonitoringStations.txt, the program emits one pair
        // key = stationId
        // value = 1 if PM2.5<=PM2.5th_limit
        //       = 0 if PM2.5>PM2.5th_limit
        JavaPairRDD<String,Integer> StationsCountersRDD=readingsRDD.mapToPair(new StationCounters(pm25th_limit));
        // Count the total number of lines and the number of lines with PM2.5>=PM2.5th_limit
        // for each station
        JavaPairRDD<String,Integer> TotalStationsCountersRDD=StationsCountersRDD.reduceByKey(new Sum());
        // Select the stations with
        // (total number of lines)=(number of lines with PM2.5>=PM2.5th_limit)
        JavaPairRDD<String,Integer> SelectedStations=TotalStationsCountersRDD.filter(new AlwaysPolluted());
        // Select the keys and store them
        SelectedStations.keys().saveAsTextFile(outputPathPartB);
        // Close the Spark context
        sc.close();
    }
}

```

```

public class StationCounters implements PairFunction<String, String, Integer> {
    private Double PM25th_limit;
    public StationCounters(Double PM25th) {
        this.PM25th_limit=PM25th;
    }
    @Override
    public Tuple2<String, Integer> call(String reading) throws Exception {
        // station1,2016/03/07,17,15,29.2,15.1
        String[] fields=reading.split(",");
        String stationId=fields[0];
        Double PM25=Double.parseDouble(fields[5]);
        if (PM25<=PM25th_limit) {
            return new Tuple2<String, Integer>(stationId, new Integer(1));
        }
        else {
            return new Tuple2<String, Integer>(stationId, new Integer(0));
        }
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2017/06/30");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(DoubleWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
        DoubleWritable, // Input value type
        Text, // Output key type
        Text> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<DoubleWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        double highestPrice = Double.MIN_VALUE;
        double lowestPrice = Double.MAX_VALUE;
        // Iterate over the set of values and compute max and min prices
        for (DoubleWritable price : values) {
            if (price.get() > highestPrice) {
                highestPrice = price.get();
            }
            if (price.get() < lowestPrice) {
                lowestPrice = price.get();
            }
        }
        double absDiff = highestPrice - lowestPrice;
        double monthlyPercPriceVariation = 100.0 * (highestPrice - lowestPrice) / lowestPrice;
        if (monthlyPercPriceVariation > 5) {
            context.write(key,
                new Text(absDiff + ", " + monthlyPercPriceVariation));
        }
    }
}

public class FirstDatePriceLastDatePrice implements Serializable {
    public String firstdate;
    public Double firstprice;
    public String lastdate;
    public Double lastprice;
    public FirstDatePriceLastDatePrice(String firstdate, Double firstprice,
        String lastdate, Double lastprice) {

        this.firstdate = firstdate;
        this.firstprice = firstprice;
        this.lastdate = lastdate;
        this.lastprice = lastprice;
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        DoubleWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // stockId,date,hour:minute,price
        // Example: FCAU,2016/06/20,16:10,10.43
        String[] fields = value.toString().split(",");
        String stockId = fields[0];
        String date = fields[1];
        String year = date.split("/")[0];
        String month = date.split("/")[1];
        Double price = Double.parseDouble(fields[3]);
        if (year.compareTo("2016") == 0) {
            // emit the pair (stockid_month, price)
            context.write(new Text(stockId + "_" + month),
                new DoubleWritable(price));
        }
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathPrices;
        String outputPathPartA;
        String outputPathPartB;
        int nw;
        inputPathPrices = args[0];
        nw = Integer.parseInt(args[1]);
        outputPathPartA = args[2];
        outputPathPartB = args[3];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2016_06_30 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of Prices.txt
        JavaRDD<String> prices = sc.textFile(inputPathPrices);
        // Select data of year 2016
        JavaRDD<String> prices2016 = prices.filter(line -> {
            // Input format: stockId,date,hour:minute,price
            String fields[] = line.split(",");
            String date = fields[1];
            String year = date.split("/")[0];
            // Select data of year 2016
            if (year.compareTo("2016") == 0) {
                return true;
            } else {
                return false;
            }
        });
        // Extract one pair (stockId_date, price) for each input line
        JavaPairRDD<String, Double> stockDate_Price = prices2016.mapToPair(line -> {
            // Input format: stockId,date,hour:minute,price
            // Extract one pair (stockId_date, price) for each input line
            String fields[] = line.split(",");
            String stockId = fields[0];
            String date = fields[1];
            Double price = new Double(fields[3]);
            return new Tuple2<String, Double>(stockId + "_" + date, new Double(price));
        }).cache();
        // Select the lowest price for each pair (stockId, date)
        JavaPairRDD<String, Double> stockDate_LowestPrice = stockDate_Price.reduceByKey((price1, price2) -> {
            if (price1.doubleValue() < price2.doubleValue()) {
                return new Double(price1);
            } else {
                return new Double(price2);
            }
        });
        JavaPairRDD<String, Double> sortedStockDate_LowestPrice = stockDate_LowestPrice.sortByKey();
        // Store the selected users in the output folder
        sortedStockDate_LowestPrice.saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // stockDate_Price already contains one element for each pair
        // (stockId_date, price) for each input line
        // Select the highest price for each pair (stockId, date)
        JavaPairRDD<String, Double> stockDate_HighestPrice = stockDate_Price
            .reduceByKey((Double price1, Double price2) -> {
                if (price1.doubleValue() > price2.doubleValue()) {
                    return new Double(price1);
                } else {
                    return new Double(price2);
                }
            });
    }
}

```

```

    });
    // Map each input pair to a new pair
    // Input pair: (stockId_date, highest_price)
    // New pair: ((stockId_weekNumber, (firstdate_highestPrice,
    // lastdate_highestPrice))
    JavaPairRDD<String, FirstDatePriceLastDatePrice> stockId_WeekNumber_date_Price = stockDate_HighestPri
    .mapToPair((Tuple2<String, Double> input) -> {
        // Input pair: (stockId_date, highest_price)
        // New pair: ((stockId_weekNumber, date_highestPrice)
        String keys[] = input._1().split("_");
        String stockId = keys[0];
        String date = keys[1];
        Integer weekNum = DateTool.weekNumber(date);
        Double price = input._2();
        FirstDatePriceLastDatePrice dp = new
            FirstDatePriceLastDatePrice(date, price, date, price);
        return new Tuple2<String, FirstDatePriceLastDatePrice>(stockId + "_" + weekNum, dp);
    });
    // ReduceByKey to select for each key the first date (and the associated
    // highestPrice) and the last date (and the associated highestprice)
    JavaPairRDD<String, FirstDatePriceLastDatePrice> stockId_WeekNumber_firstdate_lastdate = stockId_WeekN
    .reduceByKey((FirstDatePriceLastDatePrice dp1, FirstDatePriceLastDatePrice dp2) -> {
        String firstdate;
        Double firstprice;
        String lastdate;
        Double lastprice;
        // Select the first date comparing the first dates of the two input elements
        // and the associated price
        if (dp1.firstdate.compareTo(dp2.firstdate) < 0) {
            firstdate = dp1.firstdate;
            firstprice = dp1.firstprice;
        } else {
            firstdate = dp2.firstdate;
            firstprice = dp2.firstprice;
        }
        // Select the last date comparing the last dates of the two input elements
        // and the associated price
        if (dp1.lastdate.compareTo(dp2.lastdate) > 0) {
            lastdate = dp1.lastdate;
            lastprice = dp1.lastprice;
        } else {
            lastdate = dp2.lastdate;
            lastprice = dp2.lastprice;
        }
        return new FirstDatePriceLastDatePrice(firstdate, firstprice, lastdate, lastprice);
    });
    // Select pairs associate with positive weeks
    JavaPairRDD<String, FirstDatePriceLastDatePrice> stockId_WeekNumber_PositiveWeeks = stockId_WeekNumbe
    .filter((Tuple2<String, FirstDatePriceLastDatePrice> inputPair) -> {
        // Check if the week is "positive"
        if (inputPair._2().lastprice.doubleValue() > inputPair._2().firstprice.doubleValue()) {
            return true;
        } else {
            return false;
        }
    });
    // Count the number of positive weeks for each stock
    // Map each input pair to a new pair
    // Input pair: (stockId_WeekNumber, (firstdate_highestPrice,lastdate_highestPrice)
    // New pair: ((stockId, 1)
    JavaPairRDD<String, Integer> stockId_One = stockId_WeekNumber_PositiveWeeks
    .mapToPair((Tuple2<String, FirstDatePriceLastDatePrice> inputPair) -> {
        String stockId_WeekNumber = inputPair._1();
        String fields[] = stockId_WeekNumber.split("_");
        String stockId = fields[0];
        return new Tuple2<String, Integer>(new String(stockId), new Integer(1));
    });
    // Count ones per stock
    JavaPairRDD<String, Integer> stockId_Count = stockId_One
    .reduceByKey((value1, value2) -> new Integer(value1 + value2));
    // Filter stocks base on the number of "positive weeks"
    JavaPairRDD<String, Integer> frequentStockId_Count = stockId_Count
    .filter((Tuple2<String, Integer> inputPair) -> {
        if (inputPair._2().compareTo(nw) >= 0) {
            return true;
        } else {
            return false;
        }
    });
    frequentStockId_Count.keys().saveAsTextFile(outputPathPartB);
    sc.close();
}
}

```

20170714

```
public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2017/07/14");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(NullWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }
    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
            Text, // Input value type
            Text, // Output key type
            NullWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<Text> values, // Input value type
        Context context) throws IOException, InterruptedException {
        boolean greaterThan35 = false;
        boolean lessThanMinus20 = false;
        // Iterate over the set of values and check if there is at least
        // one "g" and at least on "l"
        for (Text flags : values) {
            flags.toString().split("_");
            if (flags.toString().compareTo("g") == 0) {
                greaterThan35 = true;
            }
            if (flags.toString().compareTo("l") == 0) {
                lessThanMinus20 = true;
            }
        }
        if (greaterThan35 == true && lessThanMinus20 == true) {
            context.write(key, NullWritable.get());
        }
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
            Text, // Input value type
            Text, // Output key type
            Text> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // date_reading,city,country,max_temperature,min_temperature
        // 2016/07/20,Turin,Italy,32.5,26.0
        String[] fields = value.toString().split(",");
        String city = fields[1];
        Double maxTemp = Double.parseDouble(fields[3]);
        Double minTemp = Double.parseDouble(fields[4]);
        if (maxTemp.doubleValue() > 35) {
            // emit the pair (city, "g") -> g = greater than 35
            context.write(new Text(city), new Text("g"));
        }
        if (minTemp.doubleValue() < -20) {
            // emit the pair (city, "l") -> l = less than -20
            context.write(new Text(city), new Text("l"));
        }
    }
}

public class MaxTempCount implements Serializable{
    double maxTemp;
    int count;
    public MaxTempCount(double maxTemp, int count) {
        this.maxTemp = maxTemp;
        this.count = count;
    }
    public String toString() {
        return new String(""+maxTemp/"(double)count);
    }
}

public class CityMaxTempCount implements Serializable {
    String city;
    double maxTemp;
    int count;
    public CityMaxTempCount(String city, double maxTemp, int count) {
        this.city = city;
        this.maxTemp = maxTemp;
        this.count = count;
    }
}

public class City implements Function<Tuple2<String, Tuple2<CityMaxTempCount, MaxTempCount>>, String> {
    @Override
    public String call(Tuple2<String, Tuple2<CityMaxTempCount, MaxTempCount>> value) throws Exception {
        return new String(value._2()._1().city);
    }
}
```

```

public class CountryCityStats implements PairFunction<Tuple2<String, MaxTempCount>, String, CityMaxTempCou
@Override
public Tuple2<String, CityMaxTempCount> call(Tuple2<String, MaxTempCount> value) throws Exception {
    String fields[] = value._1().split("-");
    String city = fields[0];
    String country = fields[1];
    return new Tuple2<String, CityMaxTempCount>(new String(country),
        new CityMaxTempCount(city, value._2().maxTemp, value._2().count));
}
}

public class CountryMaxTemp implements PairFunction<Tuple2<String, MaxTempCount>, String, MaxTempCount> {
@Override
public Tuple2<String, MaxTempCount> call(Tuple2<String, MaxTempCount> cityStatistics) throws Exception {
    return new Tuple2<String, MaxTempCount>(cityStatistics._1().split("-")[1],
        new MaxTempCount(cityStatistics._2().maxTemp, cityStatistics._2().count));
}
}

public class HotCity implements Function<Tuple2<String, Tuple2<CityMaxTempCount, MaxTempCount>>, Boolean>
@Override
public Boolean call(Tuple2<String, Tuple2<CityMaxTempCount, MaxTempCount>> values) throws Exception {
    double avgCity = values._2()._1().maxTemp / (double) values._2()._1().count;
    double avgCountry = values._2()._2().maxTemp / (double) values._2()._2().count;
    if (avgCity > 5 + avgCountry) {
        return true;
    } else {
        return false;
    }
}
}

public class SumTempCount implements Function2<MaxTempCount, MaxTempCount, MaxTempCount> {
@Override
public MaxTempCount call(MaxTempCount v1, MaxTempCount v2) throws Exception {
    return new MaxTempCount(v1.maxTemp + v2.maxTemp, v1.count + v2.count);
}
}

public class YearSummer2015 implements Function<String, Boolean> {
@Override
public Boolean call(String line) throws Exception {
    // 2016/07/20,Turin,Italy,32.5,26.0
    String fields[] = line.split(",");
    String date = fields[0];
    if (date.compareTo("2015/06/01") >= 0 && date.compareTo("2015/08/31") <= 0) {
        return true;
    } else {
        return false;
    }
}
}

public class CityCountryMaxTemp implements PairFunction<String, String, MaxTempCount> {
@Override
public Tuple2<String, MaxTempCount> call(String line) throws Exception {
    // 2016/07/20,Turin,Italy,32.5,26.0
    String fields[] = line.split(",");
    String city = fields[1];
    String country = fields[2];
    double maxTemp = Double.parseDouble(fields[3]);
    return new Tuple2<String, MaxTempCount>(new String(city + "-" + country), new MaxTempCount(maxTemp, 1)
}

public class CityOne implements PairFunction<String, String, Integer> {
@Override
public Tuple2<String, Integer> call(String line) throws Exception {
    // 2016/07/20,Turin,Italy,32.5,26.0
    String fields[] = line.split(",");
    String city = fields[1];
    return new Tuple2<String, Integer>(city, new Integer(1));
}
}
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathPrices;
        String outputPathPartA;
        String outputPathPartB;
        inputPathPrices = args[0];
        outputPathPartA = args[1];
        outputPathPartB = args[2];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2016_07_14 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of Temperatures.txt
        JavaRDD<String> temperatures = sc.textFile(inputPathPrices);
        // Select data of year summer 2015
        JavaRDD<String> tempSummer2015 = temperatures.filter(new YearSummer2015());
        // Extract (city_county, maxTemp_1)
        JavaPairRDD<String, MaxTempCount> cityCountryMaxTempCount = tempSummer2015.mapToPair(new CityCountryMa
        // Sum temps and counts
        JavaPairRDD<String, MaxTempCount> cityCountrySumMaxTempCount = cityCountryMaxTempCount
            .reduceByKey(new SumTempCount()).cache();
        // Store the results of the first part
        cityCountrySumMaxTempCount.saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // Extract (country, MaxTempCount city of that country)
        JavaPairRDD<String, MaxTempCount> countryMaxTempCount = cityCountrySumMaxTempCount
            .mapToPair(new CountryMaxTemp());
        // Sum temps and counts
        JavaPairRDD<String, MaxTempCount> countrySumMaxTempCount = countryMaxTempCount.reduceByKey(new SumTemp
        // Map cityCountrySumMaxTempCount to (country, city_sumtemps_count)
        JavaPairRDD<String, CityMaxTempCount> county_CitySumMaxTempCount = cityCountrySumMaxTempCount
            .mapToPair(new CountryCityStats());
        // Join
        JavaPairRDD<String, Tuple2<CityMaxTempCount, MaxTempCount>> risJoin = county_CitySumMaxTempCount
            .join(countrySumMaxTempCount);
        // Filter average city > average country
        JavaPairRDD<String, Tuple2<CityMaxTempCount, MaxTempCount>> filteredRisJoin = risJoin.filter(new HotCi
        // Extract city
        JavaRDD<String> hotCities = filteredRisJoin.map(new City());
        hotCities.saveAsTextFile(outputPathPartB);
        sc.close();
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;

        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2017/09/14");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true) == true) exitCode = 0;
        else exitCode = 1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
        IntWritable, // Input value type
        Text, // Output key type
        DoubleWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        int numFlights = 0;
        int numCancelledFlights = 0;
        double percCancelled;
        // Iterate over the set of values and check if the flag is 1 (cancelled)
        // or 0 (not cancelled)
        for (IntWritable flag : values) {
            numFlights++;
            if (flag.get() == 1)
                numCancelledFlights++;
        }
        percCancelled = (double) numCancelledFlights / (double) numFlights;
        if (percCancelled > 0.01) {
            context.write(key, new DoubleWritable(100.0 * percCancelled));
        }
    }

    public class AirlineAirportOne implements
        PairFunction<Tuple2<String, Tuple2<String, String>>, String, Integer> {
        @Override
        public Tuple2<String, Integer> call(Tuple2<String, Tuple2<String, String>> value) throws Exception {
            // key = airport id
            // value = (airline, airport name)
            String airline = value._2()._1();
            String airportName = value._2()._2();
            return new Tuple2<String, Integer>(new String(airline + "," + airportName), new Integer(1));
        }
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // AII103,Alitalia,2016/03/01,15:35,17:10,TRN,CDG,5,no,150,143
        String[] fields = value.toString().split(",");
        String departureAirport = fields[5];
        String date = fields[2];
        String cancelled = fields[8];
        int flag;
        // Select only the flights of the last year
        if (date.compareTo("2016/09/01") >= 0 && date.compareTo("2017/08/31") <= 0) {
            // If the flight was cancelled emit 1.
            // 0 otherwise.
            if (cancelled.compareTo("yes") == 0)
                flag = 1;
            else
                flag = 0;
            // emit the pair (departure airport, flag)
            context.write(new Text(departureAirport), new IntWritable(flag));
        }
    }
}

public class Counter implements Serializable {
    public int numFullyBooked = 0;
    public int numCancelled = 0;
    public int numFlights = 0;
    public String toString() {
        return new String(numFullyBooked + " " + numCancelled + " " + numFlights);
    }
}

public class MaxTempCount implements Serializable {
    double maxTemp;
    int count;
    public MaxTempCount(double maxTemp, int count) {
        this.maxTemp = maxTemp;
        this.count = count;
    }
    public String toString() {
        return new String(" "+maxTemp/" +(double)count);
    }
}

public class Sum implements Function2<Integer, Integer, Integer> {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return new Integer(v1 + v2);
    }
}

```

```

public class ArrivalAirportAirline implements PairFunction<String, String, String> {
    @Override
    public Tuple2<String, String> call(String flight) throws Exception {
        String[] fields = flight.split(",");
        String airline = fields[1];
        String arrivalAirportId = fields[6];
        return new Tuple2<String, String>(new String(arrivalAirportId), new String(airline));
    }
}

public class ArrivalAirportIdName implements PairFunction<String, String, String> {
    @Override
    public Tuple2<String, String> call(String airport) throws Exception {
        String[] fields = airport.split(",");
        String airportId = fields[0];
        String name = fields[1];
        return new Tuple2<String, String>(new String(airportId), new String(name));
    }
}

public class GermanAirport implements Function<String, Boolean> {
    @Override
    public Boolean call(String line) throws Exception {
        String[] fields = line.split(",");
        if (fields[3].compareTo("Germany") == 0)
            return true;
        else
            return false;
    }
}

public class SumCounters implements Function2<Counter, Counter, Counter> {
    @Override
    public Counter call(Counter count1, Counter count2) throws Exception {
        Counter count = new Counter();
        count.numFullyBooked = count1.numFullyBooked + count2.numFullyBooked;
        count.numCancelled = count1.numCancelled + count2.numCancelled;
        count.numFlights = count1.numFlights + count2.numFlights;
        return count;
    }
}

public class LateFlight implements Function<String, Boolean> {
    @Override
    public Boolean call(String line) throws Exception {
        String[] fields = line.split(",");
        int delay = Integer.parseInt(fields[7]);
        if (delay >= 15)
            return true;
        else
            return false;
    }
}

public class Swap implements PairFunction<Tuple2<String, Integer>, Integer, String> {
    @Override
    public Tuple2<Integer, String> call(Tuple2<String, Integer> pair) throws Exception {
        return new Tuple2<Integer, String>(new Integer(pair._2()), new String(pair._1()));
    }
}

public class OverloadedRoute implements Function<Tuple2<String, Counter>, Boolean> {
    @Override
    public Boolean call(Tuple2<String, Counter> routeCounters) throws Exception {
        double percFullyBooked = (double) routeCounters._2().numFullyBooked /
            (double) routeCounters._2().numFlights;
        double percCancelled = (double) routeCounters._2().numCancelled /
            (double) routeCounters._2().numFlights;
        if (percFullyBooked >= 0.99 && percCancelled >= 0.05)
            return true;
        else
            return false;
    }
}

public class RouteFullCancelled implements PairFunction<String, String, Counter> {
    @Override
    public Tuple2<String, Counter> call(String flight) throws Exception {
        String[] fields = flight.split(",");
        String route = new String(fields[5] + "," + fields[6]);
        String cancelled = fields[8];
        int num_of_seats = Integer.parseInt(fields[9]);
        int num_of_booked_seats = Integer.parseInt(fields[10]);
        Counter counters = new Counter();
        if (num_of_seats == num_of_booked_seats)
            counters.numFullyBooked = 1;
        else
            counters.numFullyBooked = 0;
        if (cancelled.compareTo("yes") == 0)
            counters.numCancelled = 1;
        else
            counters.numCancelled = 0;
        counters.numFlights = 1;
        return new Tuple2<String, Counter>(route, counters);
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathFlights;
        String inputPathAirports;
        String outputPathPartA;
        String outputPathPartB;
        inputPathFlights = args[0];
        inputPathAirports = args[1];
        outputPathPartA = args[2];
        outputPathPartB = args[3];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2016_09_14 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of Flights.txt
        JavaRDD<String> flights = sc.textFile(inputPathFlights);
        // Select only the flights with delay>=15
        JavaRDD<String> lateFlights = flights.filter(new LateFlight());
        // Read the content of Airports.txt
        JavaRDD<String> airports = sc.textFile(inputPathAirports);
        // Select only the airports located in Germany
        JavaRDD<String> germanAirports = airports.filter(new GermanAirport());
        // Join late flights with the subset of selected arrival airports (it
        // means we are selecting
        // only the flights with an arrival airport located in Germany)
        // Before performing join, create the pairs
        // - (arrival airport id, airline)
        // - (airport id, airport name)
        JavaPairRDD<String, String> arrivalAirportId_Airline = lateFlights.mapToPair(new ArrivalAirportAirline());
        JavaPairRDD<String, String> airportId_Name = germanAirports.mapToPair(new ArrivalAirportIdName());
        // key = airport id
        // value = (airline, airport name)
        JavaPairRDD<String, Tuple2<String, String>> selectedFlights = arrivalAirportId_Airline.join(airportId_Name);
        // create the pairs (airline-arrival airport name, +1)
        JavaPairRDD<String, Integer> airlineAirportOne = selectedFlights.mapToPair(new AirlineAirportOne());
        // count
        JavaPairRDD<String, Integer> airlineAirportCounts = airlineAirportOne.reduceByKey(new Sum());
        // swap
        JavaPairRDD<Integer, String> countAirlineAirport = airlineAirportCounts.mapToPair(new Swap());
        // sort by decreasing number of late flights
        JavaPairRDD<Integer, String> sortedAirlineAirport = countAirlineAirport.sortByKey(false);
        // select values and store then in the first output folder
        sortedAirlineAirport.saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // Map flights to pairs
        // key: departure_airport_id,arrival_airport_id
        // value: two "counters": fully booked(0/1) - cancelled(0/1) - num
        // flights (+1)
        JavaPairRDD<String, Counter> fullyBookedOrCancelledOnes = flights.mapToPair(new RouteFullCancelled());
        // Count the number of fully booked, cancelled flights, and total
        // flights per route
        JavaPairRDD<String, Counter> fullyBookedOrCancelledCounters = fullyBookedOrCancelledOnes
            .reduceByKey(new SumCounters());
        // Select only the routes with at least 99% fully booked flights and at
        // least 5% cancelled flights
        JavaPairRDD<String, Counter> selectedRoutes = fullyBookedOrCancelledCounters.filter(new OverloadedRoute());
        selectedRoutes.keys().saveAsTextFile(outputPathPartB);
        sc.close();
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        Path outputDir2;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        outputDir2 = new Path(args[3]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2018/01/22");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true) == true) {
            Configuration conf2 = this.getConf();
            Job job2 = Job.getInstance(conf2);
            job2.setJobName("Job #2");
            FileInputFormat.addInputPath(job2, outputDir);
            FileOutputFormat.setOutputPath(job2, outputDir2);
            job2.setJarByClass(DriverBigData.class);
            job2.setInputFormatClass(TextInputFormat.class);
            job2.setOutputFormatClass(TextOutputFormat.class);
            job2.setMapperClass(MapperBigData2.class);
            job2.setMapOutputKeyClass(Text.class);
            job2.setMapOutputValueClass(IntWritable.class);
            job2.setReducerClass(ReducerBigData2.class);
            job2.setOutputKeyClass(Text.class);
            job2.setOutputValueClass(IntWritable.class);
            job2.setNumReduceTasks(1);
            if (job2.waitForCompletion(true) == true) exitCode = 0;
            else exitCode = 1;
        } else exitCode = 1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData2 extends
    Reducer<Text, // Input key type
        IntWritable, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    int highestNumPurchases;
    String bidHighestNumPurchases;
    @Override
    protected void setup(Context context) {
        highestNumPurchases = Integer.MIN_VALUE;
        bidHighestNumPurchases = null;
    }
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        int numPurchases = 0;
        String bid = new String(key.toString());
        // Iterate over the set of values and count the number of purchases for
        // this book
        for (IntWritable value : values) {
            numPurchases = numPurchases + value.get();
        }
        if (bidHighestNumPurchases == null || highestNumPurchases < numPurchases
            || (highestNumPurchases == numPurchases && bid.compareTo(bidHighestNumPurchases) < 0)) {
            bidHighestNumPurchases = bid;
            highestNumPurchases = numPurchases;
        }
    }
    @Override
    protected void cleanup(Context context) throws IOException, InterruptedException {
        context.write(new Text(bidHighestNumPurchases), new IntWritable(highestNumPurchases));
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // customer1,B1020,20170502_13:10,25.00
        String[] fields = value.toString().split(",");
        String bid = fields[1];
        String date = fields[2];
        // Select only the flights of the last year
        if (date.startsWith("2016") == true) {
            // emit the pair (departure airport, flag)
            context.write(new Text(bid), new IntWritable(1));
        }
    }
}

class MapperBigData2 extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Text(bidHighestNumPurchases), new IntWritable(highestNumPurchases));
        String[] fields = value.toString().split("\\t");
        String bid = fields[0];
        int numPurchases = Integer.parseInt(fields[1]);
        context.write(new Text(bid), new IntWritable(numPurchases));
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
        IntWritable, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    int highestNumPurchases;
    String bidHighestNumPurchases;
    @Override
    protected void setup(Context context) {
        highestNumPurchases = Integer.MIN_VALUE;
        bidHighestNumPurchases = null;
    }
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        int numPurchases = 0;
        String bid = new String(key.toString());
        // Iterate over the set of values and count the number of purchases for
        // this book
        for (IntWritable value : values) {
            numPurchases++;
        }
        if (bidHighestNumPurchases == null || highestNumPurchases < numPurchases
            || (highestNumPurchases == numPurchases && bid.compareTo(bidHighestNumPurchases) < 0)) {
            bidHighestNumPurchases = bid;
            highestNumPurchases = numPurchases;
        }
    }
    @Override
    protected void cleanup(Context context) throws IOException, InterruptedException {
        context.write(new Text(bidHighestNumPurchases), new IntWritable(highestNumPurchases));
    }
}

```

```

public class AnomalousPrice implements Function<Tuple2<String, MinMax>, Boolean> {
    @Override
    public Boolean call(Tuple2<String, MinMax> bidMinMax) throws Exception {
        if ((bidMinMax._2().max - bidMinMax._2().min) > 15)
            return true;
        else
            return false;
    }
}

public class BidPricePrice implements PairFunction<String, String, MinMax> {
    @Override
    public Tuple2<String, MinMax> call(String purchase) throws Exception {
        // customer1,B1020,20170502_13:10,25.00
        String[] fields = purchase.split(",");
        String bid = new String(fields[1]);
        double price = Double.parseDouble(fields[3]);
        MinMax pricePrice = new MinMax();
        pricePrice.min = price;
        pricePrice.max = price;
        return new Tuple2<String, MinMax>(bid, pricePrice);
    }
}

public class ExtractBid implements Function<String, String> {
    @Override
    public String call(String book) throws Exception {
        // B1020,The Body in the Library,Crime,Dodd and Company,1942
        String[] fields = book.split(",");
        return new String(fields[0]);
    }
}

public class GenreCounter implements PairFunction<String, String, Counter> {
    List<String> localNeverPurchased;
    public GenreCounter(List<String> localNeverPurchased) {
        this.localNeverPurchased = localNeverPurchased;
    }
    @Override
    public Tuple2<String, Counter> call(String book) throws Exception {
        // B1020,The Body in the Library,Crime,Dodd and Company,1942
        String[] fields = book.split(",");
        String bid = fields[0];
        String genre = fields[2];
        Counter count = new Counter();
        count.numBooks = 1;
        if (localNeverPurchased.contains(bid))
            count.numNeverPurchased = 1;
        else
            count.numNeverPurchased = 0;
        return new Tuple2<String, Counter>(genre, count);
    }
}

public class GenreTotal implements Function2<Counter, Counter, Counter> {
    @Override
    public Counter call(Counter count1, Counter count2) throws Exception {
        Counter count = new Counter();
        count.numBooks = count1.numBooks + count2.numBooks;
        count.numNeverPurchased = count1.numNeverPurchased + count2.numNeverPurchased;
        return count;
    }
}

public class Counter implements Serializable {
    public int numBooks;
    public int numNeverPurchased;
    public String toString() {
        return new String(""+100.0*(double)numNeverPurchased/(double)numBooks);
    }
}

public class ExtractBidPurchases implements Function<String, String> {
    @Override
    public String call(String purchase) throws Exception {
        // customer1,B1020,20170502_13:10,25.00
        String[] fields = purchase.split(",");
        return new String(fields[1]);
    }
}

public class MinMax implements Serializable {
    public double min = Double.MAX_VALUE;
    public double max = Double.MIN_VALUE;
    public String toString() {
        return new String(max + ", " + min);
    }
}

public class Purchases2017 implements Function<String, Boolean> {
    @Override
    public Boolean call(String purchase) throws Exception {
        // customer1,B1020,20170502_13:10,25.00
        String[] fields = purchase.split(",");
        if (fields[2].startsWith("2017") == true)
            return true;
        else
            return false;
    }
}

public class ReduceMinMax implements Function2<MinMax, MinMax, MinMax> {
    @Override
    public MinMax call(MinMax prices1, MinMax prices2) throws Exception {
        MinMax newMinMax = new MinMax();
        if (prices1.min < prices2.min)
            newMinMax.min = prices1.min;
        else
            newMinMax.min = prices2.min;
        if (prices1.max > prices2.max)
            newMinMax.max = prices1.max;
        else
            newMinMax.max = prices2.max;
        return newMinMax;
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathBooks;
        String inputPathPurchases;
        String outputPathPartA;
        String outputPathPartB;
        inputPathBooks = args[0];
        inputPathPurchases = args[1];
        outputPathPartA = args[2];
        outputPathPartB = args[3];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2018_01_22 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of Purchases.txt
        JavaRDD<String> purchases = sc.textFile(inputPathPurchases);
        // Select purchases 2017
        JavaRDD<String> purchases2017 = purchases.filter(new Purchases2017()).cache();
        // Compute max and min price for each book
        JavaPairRDD<String, MinMax> bidPrice = purchases2017.mapToPair(new BidPricePrice());
        JavaPairRDD<String, MinMax> bibMinMax = bidPrice.reduceByKey(new ReduceMinMax());
        // Select books having max-min>15
        JavaPairRDD<String, MinMax> booksAnomalousPrice = bibMinMax.filter(new AnomalousPrice());
        booksAnomalousPrice.saveAsTextFile(outputPathPartA);
        // *****
        // Exercise 2 - Part B
        // *****
        // Read the content of Books.txt
        JavaRDD<String> books = sc.textFile(inputPathBooks);
        // Select BIDS of all books
        JavaRDD<String> bids = books.map(new ExtractBid());
        // Select BIDS of the books with at least one purchase in 2017
        JavaRDD<String> bidsAtLeastOnePurchase = purchases2017.map(new ExtractBidPurchases());
        // Select the bids of the never purchased books (in 2017)
        JavaRDD<String> neverPurchased = bids.subtract(bidsAtLeastOnePurchase);
        // The number of never purchased books in 2017 is small enough to be
        // stored in a local variable (this assumption is reported in the
        // problem specification)
        List<String> localNeverPurchased = neverPurchased.collect();
        // Map each book to Pair(genre, (+1,0/1))
        // 1 = never purchased book 2017
        // 0 = purchased book 2017
        JavaPairRDD<String, Counter> genreCounter = books.mapToPair(new GenreCounter(localNeverPurchased));
        // Count the number of total and never purchased books for each genre
        JavaPairRDD<String, Counter> genreTotalCounts = genreCounter.reduceByKey(new GenreTotal());
        genreTotalCounts.saveAsTextFile(outputPathPartB);
        sc.close();
    }
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2018/06/26");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(NullWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true) == true) exitCode = 0;
        else exitCode = 1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // Timestamp, VSID, CPUUtilization%, RAMUtilization%
        // Timestamp format: yyyy/mm/dd_hh:mm
        // Example: 2018/05/01,15:40,VS1,10.5,0.5
        String[] fields = value.toString().split(",");
        String date = fields[0];
        String time = fields[1];
        String vsid = fields[2];
        Double cpuUtil = Double.parseDouble(fields[3]);
        String[] yyyyymmdd = date.split("/");
        String year = yyyyymmdd[0];
        String month = yyyyymmdd[1];
        int hour = Integer.parseInt(time.split(":")[0]);
        // Select only May 2018 from 9:00 to 17:59 and the lines with
        // CPUUtilization% > 99.8
        if (year.compareTo("2018") == 0 && month.compareTo("05") == 0 && hour >= 9 && hour <= 17 && cpuUtil > 99.8)
            // emit the pair (vsid, 1)
            context.write(new Text(vsid), new IntWritable(1));
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
        IntWritable, // Input value type
        Text, // Output key type
        NullWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        int sum = 0;
        // Iterate over the set of values and sum them
        for (IntWritable one : values) {
            sum = sum + one.get();
        }
        if (sum >= 10000) {
            context.write(key, NullWritable.get());
        }
    }
}

public class Counter implements Serializable {
    private int count;
    private double sumCPUUtil;
    private double sumRAMUtil;
    public Counter(int count, double sumCPUUtil, double sumRAMUtil) {
        this.count = count;
        this.sumCPUUtil = sumCPUUtil;
        this.sumRAMUtil = sumRAMUtil;
    }
}

public class CounterHighLowCPU implements Serializable {
    private int highCPU;
    private int lowCPU;
    public CounterHighLowCPU(int highCPU, int lowCPU) {
        this.highCPU = highCPU;
        this.lowCPU = lowCPU;
    }
    public String toString() {
        return new String(highCPU + "_" + lowCPU);
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        double CPUthr;
        double RAMthr;
        String inputPathPrices;
        String outputPathPartA;
        String outputPathPartB;
        CPUthr = Double.parseDouble(args[0]);
        RAMthr = Double.parseDouble(args[1]);
        inputPathPrices = args[2];
        outputPathPartA = args[3];
        outputPathPartB = args[4];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2018_06_26 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of PerformanceLog.txt
        JavaRDD<String> statistics = sc.textFile(inputPathPrices);
        // Select data related to May 2018
        // Example: 2018/05/01,15:40,VS1,10.5,0.5
        JavaRDD<String> statisticsMay2018 = statistics.filter(line -> {
            if (line.startsWith("2018/05") == true)
                return true;
            else
                return false;
        }).cache();
        // Map each input line to a pair
        // key = VSID_Hour
        // value = (1, CPUUtilization, RAMUtilization)
        JavaPairRDD<String, Counter> vsidHourCounter = statisticsMay2018.mapToPair(line -> {
            String[] fields = line.split(",");
            String time = fields[1];
            String hour = time.split(":")[0];
            String vsid = fields[2];
            double cpuUtil = Double.parseDouble(fields[3]);
            double ramUtil = Double.parseDouble(fields[4]);
            Counter counter = new Counter(1, cpuUtil, ramUtil);
            return new Tuple2<String, Counter>(new String(vsid + "_" + hour), counter);
        });
        // Sum the three part of the Counter objects
        // Sum count
        // Sum CPUUtilization
        // Sum RAMUtilization
        JavaPairRDD<String, Counter> vsidHourAvg = vsidHourCounter
            .reduceByKey((Counter c1, Counter c2) -> new Counter(c1.getCount() + c2.getCount(),
                c1.getSumCPUUtil() + c2.getSumCPUUtil(), c1.getSumRAMUtil() + c2.getSumRAMUtil()));
        // Filter pairs by applying the two thresholds
        JavaPairRDD<String, Counter> selectedVsidHour = vsidHourAvg.filter(pair -> {
            double avgCPU = pair._2().getSumCPUUtil() / (double) pair._2().getCount();
            double avgRAM = pair._2().getSumRAMUtil() / (double) pair._2().getCount();
            if (avgCPU > CPUthr && avgRAM > RAMthr)
                return true;
            else
                return false;
        });
        selectedVsidHour.keys().saveAsTextFile(outputPathPartA);
    }
}

```

```

// *****
// Exercise 2 - Part B
// *****
// Map each input line to a pair
// key = VSID+date+hour
// value = CPUUtilization
// Example of input line: 2018/05/01,15:40,VS1,10.5,0.5
JavaPairRDD<String, Double> vsidDateHourCPUUtil = statisticsMay2018.mapToPair(line -> {
    String[] fields = line.split(",");
    String date = fields[0];
    String time = fields[1];
    String hour = time.split(":")[0];
    String vsid = fields[2];
    Double cpuUtil = new Double(fields[3]);
    return new Tuple2<String, Double>(new String(vsid + "_" + date + "_" + hour), cpuUtil);
});
// Compute max CPUUtilization for each key (VSID+date+hour)
JavaPairRDD<String, Double> vsidDateHourMaxCPUUtil = vsidDateHourCPUUtil.reduceByKey((cpuUtil1, cpuUtil2) -
    if (cpuUtil1 > cpuUtil2)
        return cpuUtil1;
    else
        return cpuUtil2;
});
// Select only the lines with maxCPUUtilization>90 or <10
JavaPairRDD<String, Double> vsidDateHourMaxCPUHighLow = vsidDateHourMaxCPUUtil.filter(pair -> {
    Double maxCPUUtil = pair._2();
    if (maxCPUUtil > 90.0 || maxCPUUtil < 10.0)
        return true;
    else
        return false;
});
// Return one pair of each input element
// key = VSID+date
// value = (>90, <10)
JavaPairRDD<String, CounterHighLowCPU> vsidDateHighLow = vsidDateHourMaxCPUHighLow.mapToPair(pair -> {
    String[] fields = pair._1().split("_");
    String vsid = fields[0];
    String date = fields[1];
    Double maxCPUUtil = pair._2();
    if (maxCPUUtil > 90.0)
        return new Tuple2<String, CounterHighLowCPU>(new String(vsid + "_" + date),
            new CounterHighLowCPU(1, 0));
    else
        return new Tuple2<String, CounterHighLowCPU>(new String(vsid + "_" + date),
            new CounterHighLowCPU(0, 1));
});
// Compute how many hours with maxCPUUtilization>90 and how many hours
// with maxCPUUtilization<10
// Sum the two counters of the CounterHighLowCPU objects
JavaPairRDD<String, CounterHighLowCPU> vsidDateSumHighLow = vsidDateHighLow.reduceByKey(
    (CounterHighLowCPU c1, CounterHighLowCPU c2) -> new CounterHighLowCPU(c1.getHighCPU() + c2.getHighCPU()
        + c1.getLowCPU() + c2.getLowCPU()));
// Select only the pairs VSID+date for which
// (Num. of hours with max CPU utilization greater than 90.0) >= 10
// hours
// (Num. of hours with max CPU utilization less than 10.0) >= 10 hours
JavaPairRDD<String, CounterHighLowCPU> selectedVsidDateSumHighLow = vsidDateSumHighLow.filter(pair -> {
    CounterHighLowCPU counterHighLow = pair._2();
    if (counterHighLow.getHighCPU() >= 8 && counterHighLow.getLowCPU() >= 8)
        return true;
    else
        return false;
});
// Save the selected pairs (VSID+Date)
selectedVsidDateSumHighLow.keys().saveAsTextFile(outputPathPartB);
sc.close();
}
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numberOfReducers;
        int exitCode;
        numberOfReducers = Integer.parseInt(args[0]);
        inputPath = new Path(args[1]);
        outputDir = new Path(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2018/07/16");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(NullWritable.class);
        job.setNumReduceTasks(numberOfReducers);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData extends
    Reducer<Text, // Input key type
            IntWritable, // Input value type
            Text, // Output key type
            NullWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        Boolean hardDriveFailure = false;
        Boolean RAMFailure = false;
        // Iterate over the set of values and check if there is at least one
        // hard drive failure and at least one RAM failure
        for (IntWritable failureType : values) {
            // Only two types of failures are sent to the reducer
            // 1 -> hard drive failure
            // 2 -> RAM failure
            if (failureType.get() == 1)
                hardDriveFailure = true;
            else
                RAMFailure = true;
        }
        // Check if there is at least one hard drive failure and at least one
        // RAM failure
        if (hardDriveFailure == true && RAMFailure == true) {
            context.write(key, NullWritable.get());
        }
    }
}

class MapperBigData extends
    Mapper<LongWritable, // Input key type
            Text, // Input value type
            Text, // Output key type
            IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split data
        // Date,Time,SID,FailureType,Downtime
        // Example: 2016/05/01,05:40:31,S2,hard_drive,5
        String[] fields = value.toString().split(",");
        String date = fields[0];
        String sid = fields[2];
        String failureType = fields[3];
        // Select only April 2016
        if (date.startsWith("2016/04") == true) {
            // emit the pair (sid, 1) if the failure type is hard drive
            // emit the pair (sid, 2) if the failure type is RAM
            // emit nothing in the other cases
            if (failureType.compareTo("hard_drive")==0)
                context.write(new Text(sid), new IntWritable(1));
            else
                if (failureType.compareTo("RAM")==0)
                    context.write(new Text(sid), new IntWritable(2));
        }
    }
}

public class NumFailuresDowntime implements Serializable {
    private int numFailures;
    private int downtimeMinutes;
    public NumFailuresDowntime(int numFailures, int downtimeMinutes) {
        this.numFailures = numFailures;
        this.downtimeMinutes = downtimeMinutes;
    }
    public String toString() {
        return new String("num. failures:" + numFailures + " tot. downtime:" + downtimeMinutes);
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathServers;
        String inputPathFailures;
        String outputPathPartA;
        String outputPathPartB;
        inputPathServers = args[0];
        inputPathFailures = args[1];
        outputPathPartA = args[2];
        outputPathPartB = args[3];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2018_07_16 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of servers.txt
        JavaRDD<String> servers = sc.textFile(inputPathServers);
        // Map each input line to a pair
        // key = SID
        // value = DataCenterID
        JavaPairRDD<String, String> sidDataCenter = servers.mapToPair(line -> {
            // SID,IP,DataCenterID
            String[] fields = line.split(",");
            String sid = fields[0];
            String dataCenterId = fields[2];
            return new Tuple2<String, String>(sid, dataCenterId);
        });
        // Read the content of Failures.txt
        JavaRDD<String> failures = sc.textFile(inputPathFailures);
        // Select data related to year 2017
        // Date,Time,SID,FailureType,Downtime
        // Example: 2016/05/01,05:40:31,S2,hard_drive,5
        JavaRDD<String> failures2017 = failures.filter(line -> {
            if (line.startsWith("2017") == true)
                return true;
            else
                return false;
        }).cache();
        // Map each input line to a pair
        // key = SID
        // value = 1
        JavaPairRDD<String, Integer> sidOne = failures2017.mapToPair(line -> {
            // Date,Time,SID,FailureType,Downtime
            String[] fields = line.split(",");
            String sid = fields[2];
            return new Tuple2<String, Integer>(sid, 1);
        });
        // Compute the number of failures per SID.
        // Those values can be used to compute the total number of failures per data center
        // This pre-computation allows reducing the number of input records of the next join
        JavaPairRDD<String, Integer> sidNumFailures = sidOne.reduceByKey((v1, v2) -> v1 + v2);
        // Join sidDataCenter and sidNumFailures
        // Returned pairs
        // key = SID
        // value = Tuple2<DataCenterID, num. failures for the server>
        JavaPairRDD<String, Tuple2<String, Integer>> sidDataCenterIdFailuresServer = sidDataCenter.join(sidNumFailures);
        // Map each input element to a pair
        // key = DataCenterId
        // value = num. failures for the server
        JavaPairRDD<String, Integer> dataCenterIdSIDsFailures = sidDataCenterIdFailuresServer
            .mapToPair(pair -> new Tuple2<String, Integer>(pair._2()._1(), pair._2()._2()));
        // Count the number of failures for each dataCenterId
        JavaPairRDD<String, Integer> dataCenterIdnumFailures = dataCenterIdSIDsFailures.reduceByKey((v1, v2) -> v1 + v2);
        // Select only the data center with at least 365 failures
        JavaPairRDD<String, Integer> selectedDataCenterIdnumFailures = dataCenterIdnumFailures.filter(element -> {
            if (element._2() >= 365)
                return true;
            else
                return false;
        });
    }
}

```

```

        return true;
    else
        return false;
    });
selectedDataCenterIdnumFailures.saveAsTextFile(outputPathPartA);
// *****
// Exercise 2 - Part B
// *****
// Map each input line to a pair
// key = (SID_month)
// value = (1,Downtime)
JavaPairRDD<String, NumFailuresDowntime> sidMonthStatistics = failures2017.mapToPair(line -> {
    // Date,Time,SID,FailureType,Downtime
    String[] fields = line.split(",");
    String sid = fields[2];
    int downtime = Integer.parseInt(fields[4]);
    String month = fields[0].split("/")[1];
    return new Tuple2<String, NumFailuresDowntime>(new String(sid + "_" + month),
        new NumFailuresDowntime(1, downtime));
});
// Count the number of failures in each month for each each server
// Sum also the value of downtime to create a
// preaggregation/precomputation of
// total downtime for each server
// For each SID+month we have the total number of failures and the
// downtime minutes in that month for that server
JavaPairRDD<String, NumFailuresDowntime> sidMonthAggregateStatistics = sidMonthStatistics
    .reduceByKey((v1, v2) -> new NumFailuresDowntime(v1.getNumFailures() + v2.getNumFailures(),
        v1.getDowntimeMinutes() + v2.getDowntimeMinutes()));
// Select only the month with at least two failures
JavaPairRDD<String, NumFailuresDowntime> sidMonthAggregateStatisticsSelected = sidMonthAggregateStatistics
    .filter(pair -> {
        if (pair._2().getNumFailures() >= 2)
            return true;
        else
            return false;
    });
// Map each input element to a pair
// key = SID
// value = (1, partialDowntime)
JavaPairRDD<String, NumFailuresDowntime> sidPairs = sidMonthAggregateStatisticsSelected.mapToPair(element -
    String sid = element._1().split("_")[0];
    int downtimeMonth = element._2().getDowntimeMinutes();
    return new Tuple2<String, NumFailuresDowntime>(sid, new NumFailuresDowntime(1, downtimeMonth));
});
// Apply reduceByKey to compute the total number of months with at least
// two failures for each server
// and also the total downtime minutes for each server
JavaPairRDD<String, NumFailuresDowntime> sidNumMonthsWithFailuresAndTotalDowntime = sidPairs
    .reduceByKey((v1, v2) -> new NumFailuresDowntime(v1.getNumFailures() + v2.getNumFailures(),
        v1.getDowntimeMinutes() + v2.getDowntimeMinutes()));
// Select only the servers with 12 months with at least two failures
// and at least 1440 minutes of total downtime
JavaPairRDD<String, NumFailuresDowntime> selectedServers = sidNumMonthsWithFailuresAndTotalDowntime
    .filter(element -> {
        int monthsWith2Failures = element._2().getNumFailures();
        int downtimeMinutes2017 = element._2().getDowntimeMinutes();
        if (monthsWith2Failures == 12 && downtimeMinutes2017 > 1440)
            return true;
        else
            return false;
    });
// Save the SIDs of the selected servers
selectedServers.keys().saveAsTextFile(outputPathPartB);
sc.close();
}
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int exitCode;
        inputPath = new Path(args[0]);
        outputDir = new Path(args[1]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2019/09/03");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(NullWritable.class);
        job.setMapOutputValueClass(Text.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(DoubleWritable.class);
        job.setOutputValueClass(Text.class);
        job.setNumReduceTasks(1);
        if (job.waitForCompletion(true)==true) exitCode=0;
        else exitCode=1;
        return exitCode;
    }
    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData
        System.exit(res);
    }
}

class MapperBigData extends Mapper<LongWritable, // Input key type
    Text, // Input value type
    NullWritable, // Output key type
    Text> { // Output value type
    double highestPrice = Double.MIN_VALUE;
    String firstTimestampHighestPrice = null;
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split value
        // stockId,date,hour:minute,price
        // Example: GOOG,2015/05/21,15:05,45.32
        String[] fields = value.toString().split(",");
        String stockId = fields[0];
        String date = fields[1];
        String hourAndMinute = fields[2];
        double price = Double.parseDouble(fields[3]);
        // Select only stockId GOOG and year 2017
        if (date.startsWith("2017") == true && stockId.equals("GOOG")) {
            if (price >= highestPrice) {
                // Update highestPrice
                highestPrice = price;
                String currentTimestamp = new String(date + "_" + hourAndMinute);
                // Check if also the date must be updated
                if (firstTimestampHighestPrice == null ||
                    (currentTimestamp.compareTo(firstTimestampHighestPrice) < 0))
                    firstTimestampHighestPrice = currentTimestamp;
            }
        }
    }
    protected void cleanup(Context context) throws IOException, InterruptedException {
        // emit the local top 1 price and the associated date
        if (firstTimestampHighestPrice!=null)
            context.write(NullWritable.get(), new Text (highestPrice+","+firstTimestampHighestPrice));
    }
}

class ReducerBigData extends Reducer<NullWritable, // Input key type
    Text, // Input value type
    DoubleWritable, // Output key type
    Text> { // Output value type
    @Override
    protected void reduce(NullWritable key, // Input key type
        Iterable<Text> values, // Input value type
        Context context) throws IOException, InterruptedException {
        double highestPrice = Double.MIN_VALUE;
        String firstTimestampHighestPrice = null;
        // Iterate over the set of values and compute the global
        // maximum price and the first date it occurs
        for (Text value : values) {
            // Split value
            // localmaximumprice,timestamp
            String[] fields = value.toString().split(",");
            double price = Double.parseDouble(fields[0]);
            String timestamp = fields[1];
            if (price >= highestPrice) {
                // Update highestPrice
                highestPrice = price;
                String currentTimestamp = timestamp;
                // Check if also the date must be updated
                if (firstTimestampHighestPrice == null ||
                    (currentTimestamp.compareTo(firstTimestampHighestPrice) < 0))
                    firstTimestampHighestPrice = currentTimestamp;
            }
        }
        // Emit the result
        context.write(new DoubleWritable(highestPrice), new Text(firstTimestampHighestPrice));
    }
}

public class MinMaxPrices implements Serializable {
    private double minPrice;
    private double maxPrice;
    public MinMaxPrices(double minPrice, double maxPrice) {
        this.minPrice = minPrice;
        this.maxPrice = maxPrice;
    }
    public String toString() {
        return new String(minPrice+" "+maxPrice);
    }
}

public class StockDate implements Serializable {
    private String stockId;
    private String date;
    public StockDate(String stockId, String date) {
        this.stockId=stockId;
        this.date=date;
    }
    public String toString() {
        return new String(stockId + "," + date);
    }
    // The objects of this class are used as keys of JavaPairRDDs. Hence, custom versions of the methods
    public boolean equals(Object o2) {
        String v1 = new String(stockId + date);
        String v2 = new String(((StockDate)o2).getStockId() + ((StockDate)o2).getDate());
        if (v1.compareTo(v2) == 0)
            return true;
        else
            return false;
    }
    @Override
    public int hashCode() {
        return (new String(stockId + date)).hashCode();
    }
}

```

```

public class SparkDriver {
    public static void main(String[] args) {
        String inputPathPrices;
        String outputPathPartA;
        String outputPathPartB;
        inputPathPrices = args[0];
        outputPathPartA = args[1];
        outputPathPartB = args[2];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2018_09_03 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        // Read the content of stocks_prices.txt
        JavaRDD<String> stockPrices = sc.textFile(inputPathPrices);
        // Select only year 2017 data
        JavaRDD<String> stockPrices2017 = stockPrices.filter(line -> {
            String[] fields = line.split(",");
            String date = fields[1];
            if (date.startsWith("2017") == true)
                return true;
            else
                return false;
        });
        // Generate for each input line (of year 2017) a pair with
        // key = stockId,Date (object of type StockDate)
        // value = price,price (object of type MinMaxPrices)
        JavaPairRDD<StockDate, MinMaxPrices> stockDate_Price = stockPrices2017.mapToPair(line -> {
            String[] fields = line.split(",");
            String stockId = fields[0];
            String date = fields[1];
            double price = Double.parseDouble(fields[3]);
            StockDate key = new StockDate(stockId, date);
            MinMaxPrices value = new MinMaxPrices(price, price);
            return new Tuple2<StockDate, MinMaxPrices>(key, value);
        });
        // Compute max price and min price for each stockId+date
        JavaPairRDD<StockDate, MinMaxPrices> stockDate_MinandMaxPrice = stockDate_Price.reduceByKey((v1, v2) -> {
            double minPrice;
            double maxPrice;
            if (v1.getMinPrice() < v2.getMinPrice())
                minPrice = v1.getMinPrice();
            else
                minPrice = v2.getMinPrice();
            if (v1.getMaxPrice() > v2.getMaxPrice())
                maxPrice = v1.getMaxPrice();
            else
                maxPrice = v2.getMaxPrice();
            MinMaxPrices value = new MinMaxPrices(minPrice, maxPrice);
            return value;
        });
        // Compute daily variation for each stockId+date
        JavaPairRDD<StockDate, Double> stockDate_DailyVariation = stockDate_MinandMaxPrice
            .mapValues(minmax -> new Double(minmax.getMaxPrice() - minmax.getMinPrice())).cache();
        // Select only the element with a daily variation > 10
        JavaPairRDD<StockDate, Double> stockDate_DailyVariationGreater10 = stockDate_DailyVariation
            .filter(pairVariation -> pairVariation._2() > 10);
        // Count the number of dates with daily variation > 10 for each stockId
        // Map each input element to a pair (stockId, +1)
        JavaPairRDD<String, Integer> stockIdOne = stockDate_DailyVariationGreater10
            .mapToPair(stockDateVariation -> new Tuple2<String, Integer>(stockDateVariation._1().getStockId(),
                new Integer(1)));
        // ReduceByKey to count the number of dates for each stockId
    }
}

```

```

JavaPairRDD<String, Integer> stockIdNumDates = stockIdOne.reduceByKey((v1, v2) -> v1 + v2);
// Save result
stockIdNumDates.saveAsTextFile(outputPathPartA);
// *****
// Exercise 2 - Part B
// *****
// Consider stockDate_DailyVariation
// It contains for each stockid+Date the daily variation.
// Each element of stockDate_DailyVariation is the first date of a
// sequence of two dates and also the second date of another sequence of
// two dates.
// Emit for each input element (stockid+Date,DailyVariation) two pairs:
// 1 - key=stockId+Date - value=DailyVariation at date Date -> This is the first
// element of the sequence of two dates starting at date "Date" associated with stockId.
// 2 - key=stockId+(Date-1) - value=DailyVariation at date Date -> This is the
// second element of the sequence of two dates starting at date "Date-1" associated with stockId
JavaPairRDD<StockDate, Double> stockIdSequenceFirstDate_DailyVariations = stockDate_DailyVariation
    .flatMapToPair(stockIdDateDailyVariation -> {
        String stockId = stockIdDateDailyVariation._1().getStockId();
        String date = stockIdDateDailyVariation._1().getDate();
        String previousDate = DateTool.previousDate(date);
        Double dailyVariation = new Double(stockIdDateDailyVariation._2());
        ArrayList<Tuple2<StockDate, Double>> returnedPairs = new ArrayList<Tuple2<StockDate, Double>>();
        StockDate stockIdDate = new StockDate(stockId, date);
        StockDate stockIdPreviousDate = new StockDate(stockId, previousDate);
        // Add the pair key=stockId+Date - value=Date+DailyVariation
        returnedPairs.add(new Tuple2<StockDate, Double>(stockIdDate, dailyVariation));
        // Add the pair key=stockId+(Date-1) -
        // value=Date+DailyVariation
        returnedPairs
            .add(new Tuple2<StockDate, Double>(stockIdPreviousDate, dailyVariation));
        return returnedPairs.iterator();
    });
// Apply groupByKey to generate all the "sequences" of two dates for
// each stock with the associated dailyVariation values
// Each returned pair has the following content:
// - key = StockId+first date of the sequence of two dates
// - value = daily variations for stockId for the two dates associated with this sequence of two dates
JavaPairRDD<StockDate, Iterable<Double>> stockIdTwoConsecutiveDatesDailyVariations = stockIdSequenceFirstDa
    .groupByKey();
// Select only the "stable trends"
JavaPairRDD<StockDate, Iterable<Double>> stockIdStableTrends = stockIdTwoConsecutiveDatesDailyVariations
    .filter(inputSequence -> {
        // Select the two daily variations and compute the absolute
        // difference between those two values
        Double dailyVariation1 = null;
        Double dailyVariation2 = null;
        // Iterate over the values and store the two daily values
        for (Double value : inputSequence._2()) {
            if (dailyVariation1 == null) { // This is the first
                // value of the iterable
                dailyVariation1 = new Double(value);
            } else { // There are at most two values. This is the
                // second one
                dailyVariation2 = new Double(value);
            }
        }
        // Check if there are at two daily variation values and if
        // their absolute difference is at most 0.1.
        if (dailyVariation2 != null
            && Math.abs(dailyVariation1 - dailyVariation2) <= 0.1)
            return true;
        else
            return false;
    });
// Select the keys and store them
stockIdStableTrends.keys().saveAsTextFile(outputPathPartB);
sc.close();
}
}

```

```

public class DriverBigData extends Configured implements Tool {
    @Override
    public int run(String[] args) throws Exception {
        Path inputPath;
        Path outputDir;
        int numReducers;
        int exitCode;
        inputPath = new Path(args[0]);
        outputDir = new Path(args[1]);
        numReducers = Integer.parseInt(args[2]);
        Configuration conf = this.getConf();
        Job job = Job.getInstance(conf);
        job.setJobName("Exercise #1 - Exam 2019/02/15");
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);
        job.setJarByClass(DriverBigData.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapperClass(MapperBigData.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setReducerClass(ReducerBigData.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(NullWritable.class);
        // The number of instances of the reducer can be any value >=
        job.setNumReduceTasks(numReducers);
        if (job.waitForCompletion(true) == true) exitCode = 0;
        else exitCode = 1;
        return exitCode;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new DriverBigData(), args);
        System.exit(res);
    }
}

class ReducerBigData extends Reducer<Text, // Input key type
    IntWritable, // Input value type
    Text, // Output key type
    NullWritable> { // Output value type
    @Override
    protected void reduce(Text key, // Input key type
        Iterable<IntWritable> values, // Input value type
        Context context) throws IOException, InterruptedException {
        // Iterate over the set of values and compute
        // - total number of "tourism" POIs = number of input values
        // - total number of "museum" POIs = number of ones = sum of values
        int numTourismPOIs = 0;
        int numMuseumPOIs = 0;
        for (IntWritable value : values) {
            numTourismPOIs++;
            numMuseumPOIs = numMuseumPOIs + value.get();
        }
        // Emit the city only if
        // - total number of "tourism" POIs > 1000
        // - total number of "museum" POIs >= 20
        if (numTourismPOIs > 1000 && numMuseumPOIs >= 20)
            context.write(new Text(key), NullWritable.get());
    }
}

class MapperBigData extends Mapper<LongWritable, // Input key type
    Text, // Input value type
    Text, // Output key type
    IntWritable> { // Output value type
    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException, InterruptedException {
        // Split record
        // POI_ID,latitude,longitude,city,country,category,subcategory
        // Example: P101,45.0621644,7.578633,Turin,Italy,shop,shoes
        String[] fields = value.toString().split(",");
        String city = fields[3];
        String country = fields[4];
        String category = fields[5];
        String subcategory = fields[6];
        // Select only POIs of the Italian cities
        // and only "tourism" POIs
        if (country.equals("Italy") == true && category.equals("tourism")) {
            // Check the subcategory
            if (subcategory.contentEquals("museum")) {
                // emit pair (city, 1)
                // -- One new tourism POI
                // -- One new museum POI
                context.write(new Text(city), new IntWritable(1));
            } else {
                // emit pair (city, 0)
                // -- One new tourism POI
                // -- This POI is not a museum POI
                context.write(new Text(city), new IntWritable(0));
            }
        }
    }
}

public class CountTaxiBusstop implements Serializable {
    private int numTaxiPOIs;
    private int numBusstopPOIs;
    public CountTaxiBusstop(int numTaxiPOIs, int numBusstopPOIs) {
        this.numTaxiPOIs = numTaxiPOIs;
        this.numBusstopPOIs = numBusstopPOIs;
    }
    public String toString() {
        return new String(this.numTaxiPOIs + " " + this.numBusstopPOIs);
    }
}

public class CountCityMuseum implements Serializable {
    private int numCities;
    private int numMuseumPOIs;
    public CountCityMuseum(int numCities, int numMuseumPOIs) {
        this.setNumCities(numCities);
        this.setNumMuseumPOIs(numMuseumPOIs);
    }
    public String toString() {
        return new String(this.numCities + " " + this.numMuseumPOIs);
    }
}

```

```

public class SparkDriver {

    public static void main(String[] args) {
        String inputPathPOIs;
        String outputPathPartA;
        String outputPathPartB;
        inputPathPOIs = args[0];
        outputPathPartA = args[1];
        outputPathPartB = args[2];
        // Create a configuration object and set the name of the application
        SparkConf conf = new SparkConf().setAppName("Spark Exam 2019_02_15 - Exercise #2");
        // Create a Spark Context object
        JavaSparkContext sc = new JavaSparkContext(conf);
        // *****
        // Exercise 2 - Part A
        // *****
        /*
        * Italian cities with taxis but without buses. Considering only the Italian
        * cities, the application must select the Italian cities with at least one
        * "taxi" POI (subcategory="Taxi") but without "Busstop" POIs
        * (subcategory="Busstop"). The application stores in the first HDFS output
        * folder the selected cities, one city per line.
        */
        // Read the content of POIs.txt
        // POI_ID,latitude,longitude,city,country,category,subcategory
        JavaRDD<String> pois = sc.textFile(inputPathPOIs);
        // Select only the Italian cities
        JavaRDD<String> poisItaly = pois.filter(line -> {
            String[] fields = line.split(",");
            String country = fields[4];
            if (country.equals("Italy") == true)
                return true;
            else
                return false;
        }).cache();
        // Select only the POIs with subcategory="Taxi" or subcategory="Busstop"
        JavaRDD<String> poisItalySelectedSubcategories = poisItaly.filter(line -> {
            String[] fields = line.split(",");
            String subcategory = fields[6];
            if (subcategory.equals("taxi") == true || subcategory.equals("busstop") == true)
                return true;
            else
                return false;
        });
        // Generate for each input line a pair with
        // key = city
        // value = (taxi=0/1,busstop=0/1) (object of type CountTaxiBusstop)
        JavaPairRDD<String, CountTaxiBusstop> cityTaxiBusstop = poisItalySelectedSubcategories.mapToPair(line -> {
            String[] fields = line.split(",");
            String city = fields[3];
            String subcategory = fields[6];
            CountTaxiBusstop value;
            if (subcategory.equals("taxi")) {
                // Taxi POI associated with city
                value = new CountTaxiBusstop(1, 0);
            } else {
                // Busstop POI associated with city
                value = new CountTaxiBusstop(0, 1);
            }
            return new Tuple2<String, CountTaxiBusstop>(city, value);
        });
        // For each Italian city, compute
        // - number of taxi POIs
        // - number of Busstop POIs
        JavaPairRDD<String, CountTaxiBusstop> cityNumTaxiNumBusstop = cityTaxiBusstop.reduceByKey((v1, v2) -> {
            return new CountTaxiBusstop(v1.getNumTaxiPOIs() + v2.getNumTaxiPOIs(),
                v1.getNumBusstopPOIs() + v2.getNumBusstopPOIs());
        });
        // Select only the element with
        // - number of taxi POIs >=1
    }
}

```

```

// - number of Busstop =0
JavaPairRDD<String, CountTaxiBusstop> selectedCityNumTaxiNumBusstop = cityNumTaxiNumBusstop
    .filter(cityCounterTaxiBusstop -> {
        if (cityCounterTaxiBusstop._2().getNumTaxiPOIs() >= 1
            && cityCounterTaxiBusstop._2().getNumBusstopPOIs() == 0)
            return true;
        else
            return false;
    });
// Save the selected cities = the keys of selectedCityNumTaxiNumBusstop
selectedCityNumTaxiNumBusstop.keys().saveAsTextFile(outputPathPartA);
// *****
// Exercise 2 - Part B
// *****
/*
 * Italian cities with "many" museums with respect to the other Italian cities.
 * Considering only the Italian cities, the application must select the Italian
 * cities with a number of "museum" POIs (subcategory="museum") greater than the
 * average number of "museum" POIs per city in Italy. The application stores in
 * the second HDFS output folder the selected cities, one city per line.
 */
// Count the number of museum POIs for each Italian city

// Generate for each input line a pair with
// key = city
// value = 1 if subcategory="museum"
// value = 0 if subcategory!="museum"
JavaPairRDD<String, Integer> cityMuseumPOIs = poisItaly.mapToPair(line -> {
    String[] fields = line.split(",");
    String city = fields[3];
    String subcategory = fields[6];
    if (subcategory.equals("museum") == true) { // museum POI
        return new Tuple2<String, Integer>(city, 1);
    } else { // non-museum POI
        return new Tuple2<String, Integer>(city, 0);
    }
});
// Count the number of museum POIs for each Italian city
JavaPairRDD<String, Integer> cityNumMuseumPOIs = cityMuseumPOIs.reduceByKey((v1, v2) -> v1 + v2);
// Each input element represents
// - an Italian city
// - the number of museum POIs for that city
// Emit one new element of type CountCityMuseum with
// - numCities = 1
// - numMuseumPOIs (for that city) = value of the input pair
JavaRDD<CountCityMuseum> numCitiesNumMuseumPOIs = cityNumMuseumPOIs
    .map(pair -> new CountCityMuseum(1, pair._2()));
// Compute total number of Italian cities and total number of "museum" POIs the
// Italy (i.e., in the Italian cities)
CountCityMuseum totalNumCitiesNumMuseumPOIs = numCitiesNumMuseumPOIs
    .reduce((CountCityMuseumV1, CountCityMuseumV2) -> new CountCityMuseum(
        CountCityMuseumV1.getNumCities() + CountCityMuseumV2.getNumCities(),
        CountCityMuseumV1.getNumMuseumPOIs() + CountCityMuseumV2.getNumMuseumPOIs()));
// Compute average number of "museum" POIs per city in Italy
double average = (double) totalNumCitiesNumMuseumPOIs.getNumMuseumPOIs()
    / (double) totalNumCitiesNumMuseumPOIs.getNumCities();
// Select only the Italian cities with a number of "museum" POIs
// (subcategory="museum") greater than the average number of "museum" POIs per
// city in Italy.
JavaPairRDD<String, Integer> selectedCityNumMuseumPOIs = cityNumMuseumPOIs.filter(inputPair -> {
    if (inputPair._2() > average)
        return true;
    else
        return false;
});
// Store the selected cities
selectedCityNumMuseumPOIs.keys().saveAsTextFile(outputPathPartB);
sc.close();
}
}

```